

JSONCANON: Trustworthy JSON Canonicalization via Dual Independent Implementations and Differential Testing

Luciano Federico Pereira*

June 22, 2026

Abstract

A hash, byte comparison, or `git diff` over JSON is only meaningful if the JSON has first been reduced to a single deterministic byte stream. Many canonicalizers exist, but their correctness usually rests on a single implementation and a test suite written by the same author against the same mental model. We present JSONCANON, a JSON canonicalizer and adjacent toolkit built around a different correctness discipline: *two fully independent implementations* (in Python and Nim) that are required to produce *byte-identical* output, arbitrated by a written specification and enforced by a differential-testing gate. We argue that this dual-implementation byte-parity contract, combined with validation against *external* oracles (a JavaScript engine for number formatting, CPython’s regular-expression engine for pattern matching), yields markedly higher confidence than single-implementation testing, because the two implementations share no code and were written to agree only through the specification. To our knowledge no existing tool offers this combination: a single self-contained, dependency-free package that pairs two byte-identical implementations with coverage of the full JSON-integrity lifecycle—lenient ingestion of real-world dialects, canonicalization, validation against four schema languages, deterministic binary encodings, and structural transforms—where every nontrivial component (shortest-float printing, SHA-256, regular-expression matching, the schema engines) is reimplemented in-tree rather than delegated to a third-party library. The individual capabilities exist as scattered single-language libraries; the contribution is their integration under one specification and one correctness contract. We describe the canonical form—in particular a number-normalization scheme that operates purely on decimal strings and never routes a value through a binary float—and the deliberate tension between that lossless default and RFC 8785 (the JSON Canonicalization Scheme), which *requires* IEEE-754 rounding; we resolve the tension with an opt-in mode that loudly reports every value whose meaning changes. We detail determinism-by-construction techniques (generated lookup tables shared across both languages, a shared regular-expression subset, strict per-code-point processing) and a reproducible validation methodology (millions of randomized inputs diffed against external oracles, dozens of cross-language parity assertions). The result is a single deterministic byte stream over a wide superset of real-world JSON dialects, plus interoperable validation (JSON Schema, JSON Type Definition, CDDL, GeoJSON, I-JSON), cross-format conversion (CBOR, MessagePack, JData), and structural transforms (JSON Pointer, Patch, Merge Patch). We are explicit about what the method does *not* guarantee: byte parity proves the two implementations agree, not that they are both correct—hence the external oracles—and several schema features are deliberate subsets.

Keywords: canonical JSON; deterministic serialization; differential testing; content addressing; reproducibility; RFC 8785; shortest float; data science; machine-learning data pipelines.

*Independent researcher. ORCID: 0009-0002-4591-6568. Source, specification, and reproduction harnesses are released with this paper at <https://github.com/lucianofedericopereira/json-canon>.

1 Introduction

The promise of canonical JSON is simple to state: take JSON produced by *anything*—a Python emitter, a pandas frame, a hand-edited file with comments and trailing commas, a JavaScript service—and emit one deterministic byte stream, so that `cmp`, `sha256`, and `git diff` over the output mean something regardless of the producer. The difficulty is entirely in the word “deterministic.” Producers disagree on object-member order, on whether 4.0 and 4 are the same number, on `ensure_ascii` escaping, on whitespace, and on a long tail of dialect features. A canonicalizer must collapse all of that to a single normal form, and—if its output is to be used as a content address—it must do so identically across platforms, languages, and time.

This raises a question that is rarely asked directly: *how do we know a canonicalizer is correct?* The usual answer is a test suite. But a test suite written by the implementer encodes the implementer’s understanding of the specification; a misreading of the specification appears in both the code and the tests, and passes. For a primitive whose entire value is that independent parties compute the same bytes, this is an uncomfortable foundation.

JSONCANON adopts a stronger discipline. It ships *two* implementations, in Python and in Nim, that share no source code. A written contract (`SPEC.md`) is the sole arbiter: where the two disagree, the specification wins and at least one implementation has a bug. A continuous gate asserts that the two produce byte-identical output across a corpus of fixtures and a matrix of options, and—crucially—validates the genuinely hard, non-obvious pieces (shortest-float formatting, regular-expression matching) against *external* reference implementations rather than against each other. The thesis of this paper is that this combination—*independent reimplementation for agreement, external oracles for correctness*—is a practical and effective way to build a trustworthy canonicalizer, and that the engineering techniques required to make two languages agree byte-for-byte are themselves worth reporting.

The implementations total roughly 5,600 lines of Nim and 3,700 lines of Python; the gate spans fifty-eight cross-language parity assertions and is clean under `mypy --strict`, `pyright`, and `nim check`.

Contributions.

- A *dual-implementation byte-parity contract* for canonical serialization, with a specification as arbiter and a differential gate as enforcement (Section 5).
- A lossless number-canonicalization scheme that operates on decimal strings and never touches a binary float, making cross-language agreement independent of any float printer; and an analysis of its tension with RFC 8785, resolved by a loud, non-silenceable warning discipline (Sections 4 and 7).
- *Determinism-by-construction* techniques for cross-language agreement: lookup tables generated once and embedded in both languages, a shared regular-expression subset, and strict per-code-point processing (Section 6).
- A reproducible validation methodology using external oracles and large-scale differential fuzzing (Section 8).
- A *self-contained, dependency-free* artifact that integrates the full JSON-integrity lifecycle—lenient ingestion, canonicalization, four schema languages, deterministic binary formats, and structural transforms—under a single specification and parity contract. We are not aware of another tool that combines these properties in one package (Section 3); the novelty is in the integration, the correctness discipline, and the self-containment, not in any individual algorithm.

We are equally explicit about the limitations (Section 10): parity is agreement, not correctness; some validators are subsets; there are no machine-checked proofs.

2 Scope and Applications

What the tool is for (and not for). JSONCANON is useful precisely when JSON output must be *compared, hashed, signed, deduplicated, or diffed*—that is, when the bytes, not just the parsed value, carry meaning. It is *not* a faster JSON parser, a data-processing framework, or a database. Its design centre of gravity is integrity: the default number engine is lossless (Section 4), so the canonical bytes are an exact, reversible re-presentation of the input value, suitable as a content address. When byte-level interoperability with an external ecosystem is the goal—most importantly JSON Web Signature and the broader JOSE family Jones et al. (2015), which standardize on JCS—the opt-in JCS mode produces exactly the bytes those systems expect, at the documented cost of binary64 rounding. The two regimes (lossless content addressing vs. JCS interoperability) cover most real needs, and the tool makes the choice explicit, not implicit.

Data science and pandas. Tabular workflows routinely produce “the same” table in byte-different JSON: columns in a different order, an integer column that became float after a `groupby`, or simply a different emitter. JSONCANON makes such tables hash-equal. The package ships a `pandas` McKinney (2010) accessor that serializes a `DataFrame`/`Series` through a fixed internal orientation and then canonicalizes, so two frames holding the same data share a SHA-256 regardless of column order or int-vs-float dtype (Listing 1). This underpins content-addressed caching of intermediate artifacts, deduplication of result sets, and reproducible-pipeline checks where a stage is re-run only if the canonical hash of its inputs changed.

Listing 1: Content-addressed equality of tabular data, despite column-order and dtype differences.

```
import pandas as pd, jsoncanon.pandas_accessor
df1 = pd.DataFrame({"price": [4, 10], "qty": [2, 3]}) # int64
df2 = pd.DataFrame({"qty": [2., 3.], "price": [4., 10.]}) # float64, reordered
df1.jsoncanon.sha256() == df2.jsoncanon.sha256() # True
```

Statistics and scientific computing. Numeric data exchange is where the lossless default earns its keep. High-precision decimals and integers beyond 2^{53} —survey weights, fixed-point monetary values, large identifiers, exact rationals serialized as decimals—are preserved character-for-character rather than silently rounded through a float, and the two implementations agree without depending on any platform’s float printer. For array-shaped numeric data, the JData/NeuroJSON convention Fang (2015) (used in neuroimaging and scientific computing) is decoded into plain nested arrays so that N-dimensional datasets can be canonicalized, hashed, and deduplicated like any other JSON. The result is a stable content identity for datasets that reproduces across machines and language runtimes—a precondition for citable, re-runnable computational results.

LLM and AI systems. Large-language-model systems generate and consume JSON constantly—tool-call arguments, structured outputs, evaluation records—and benefit from a canonical form in several distinct ways. (i) *Caching*: keying a prompt, tool call, or retrieval result by the SHA-256 of its canonical form means semantically identical payloads collide in the cache even when a model reorders object keys or emits 4.0 where a previous call emitted 4. (ii) *Dataset hygiene*: training and evaluation corpora can be deduplicated, and train/test contamination detected, by

canonical-hash equality rather than fragile string matching. *(iii) Structured-output validation:* model-produced JSON can be checked against a JSON Schema or JSON Type Definition before it is trusted by downstream code, with identical verdicts in whichever language the service is written in. *(iv) Reproducible evaluation:* logging canonical hashes of inputs and outputs makes eval runs comparable across time and harness versions. *(v) Integrity of identifiers:* models frequently emit large integer IDs; the JCS-mode warning discipline (Section 7) surfaces exactly the cases where naive float-based canonicalization would silently corrupt such an ID—a failure a single-implementation tool might never reveal.

Security, integrity, and reproducible builds. Canonicalization is a prerequisite for signing and content addressing. Detached signatures over JSON (JOSE/JWS Jones et al. (2015)) require a canonical byte stream; content-addressable stores, tamper-evident logs, and build/supply-chain attestations all need “same value $\Rightarrow$ same bytes $\Rightarrow$ same hash.” Here the lossless default is the safe choice: it guarantees the hashed bytes reproduce the input exactly, and the dual-implementation discipline reduces the risk that a verifier written in a different language computes a different digest.

Configuration, DevOps, and cross-language services. Canonicalizing configuration before committing or comparing turns `git diff` into a semantic diff (the `--diff` mode reports changes by JSON path, ignoring formatting and key-order noise) and makes drift between environments detectable. For polyglot service meshes, the contract that a Python service and a Nim service (or any future port adhering to the same specification) emit identical canonical bytes is exactly the property needed for cross-language caches, idempotency keys, and request signing; the CBOR and MessagePack bridges additionally let those bytes be exchanged compactly while remaining deterministic.

3 Background and Related Work

JSON and canonicalization. JSON is specified by RFC 8259 Bray (2017), which fixes a grammar but deliberately leaves member ordering and number representation to producers. Several canonical forms have been proposed to remove that latitude. The most prominent standardized one is the JSON Canonicalization Scheme (JCS), RFC 8785 Rundgren et al. (2020), which sorts object keys by UTF-16 code unit and serializes numbers exactly as ECMAScript’s `Number.prototype.toString` would. Other ad-hoc “canonical JSON” conventions (e.g. the OLPC form) predate JCS and differ mainly in number and escaping rules. JSONCANON’s default form is deliberately *not* JCS—it preserves arbitrary-precision numbers as exact decimals—and offers JCS as an explicit mode.

Shortest-float printing. JCS number output is the shortest decimal string that round-trips to the same IEEE-754 binary64, with ECMAScript’s exponent thresholds. The modern algorithm for the shortest round-tripping decimal is Ryu Adams (2018); we port its double-to-string core to both languages.

Dialects and binary encodings. Real input includes JSON5 Kishore et al. (2023) (unquoted keys, comments, hex, etc.), NDJSON, and RFC 7464 JSON text sequences Williams (2015). Binary serializations with their own determinism stories include CBOR Bormann and Hoffman (2020) (whose §4.2 defines a core deterministic encoding) and MessagePack Furuhashi (2008). JData/NeuroJSON Fang (2015) encodes N-dimensional numeric arrays in JSON.

Table 1: Positioning by capability category (illustrative, not an exhaustive feature audit; representative tools, current at time of writing). ✓ present; – absent.

Tool	Dual impl. (byte-parity)	No 3rd-party runtime deps	Canonical form	Schema languages	Binary formats	Struct. transforms
JSONCANON (this work)	✓	✓	✓	4	2	✓
JCS library (per lang.)	–	varies	✓ (JCS)	0	–	–
ajv	–	–	–	1	–	–
python-jjsonschema	–	–	–	1	–	–
cbor2 / msgpack	–	varies	–	0	1	–
json-patch	–	–	–	0	–	✓

Validation. Schema languages relevant to JSON include JSON Schema Wright et al. (2022) (Draft 2020-12 and the widely deployed Draft-07), JSON Type Definition (RFC 8927) Carion (2020), and CDDL (RFC 8610) Birkholz et al. (2019). RFC 7493 (I-JSON) Bray (2015) defines an interoperable subset, and RFC 7946 (GeoJSON) Butler et al. (2016) a geospatial profile with its own structural rules.

Pointers and patches. JSON Pointer (RFC 6901) Bryan et al. (2013), JSON Patch (RFC 6902) Bryan and Nottingham (2013), and JSON Merge Patch (RFC 7386) Hoffman and Snell (2014) address and transform documents.

Differential testing. The idea of cross-checking independent implementations to find bugs is well established in compilers and parsers McKeeman (1998); Yang et al. (2011). Our contribution is not the technique but its application as a *primary correctness contract* for a canonicalizer, combined with external oracles for the components where “agreement” alone would be insufficient.

Positioning relative to existing tools. Every capability JSONCANON provides has prior art, but—to our knowledge—only in *separate*, typically single-language, typically dependency-bearing libraries: JCS implementations exist per language; `ajv` and `python-jjsonschema` validate JSON Schema; `cbor2` and `msgpack` handle the binary formats; `python-jsonpatch` applies RFC 6902. We are not aware of any single tool that simultaneously (i) ships *two independent implementations held to byte-identical output* as its correctness contract; (ii) is *self-contained and free of third-party runtime dependencies*, reimplementing even shortest-float conversion (Ryu), SHA-256, the regular-expression engine, and the schema validators in-tree—in part precisely so that cross-language agreement does not hinge on any external library; and (iii) spans the *entire JSON-integrity lifecycle*, from lenient ingestion of real-world dialects through canonicalization, validation against four schema languages (JSON Schema, JTD, and CDDL, plus the GeoJSON and I-JSON profiles), deterministic binary re-encoding, and structural transforms, in one artifact. The novelty claimed here is this integration and the discipline behind it, not the rediscovery of any underlying algorithm. Table 1 summarizes the gap.

4 The Canonical Form

The canonical output is UTF-8 with no byte-order mark, no insignificant whitespace, separators `,` and `:`, and no trailing newline (unless requested). Three rules carry the weight.

Numbers (the important one). Numbers are normalized *as decimal strings*, using only string and integer-digit operations; a number token is never parsed into a binary float. Given a token, the implementation decomposes it into sign, integer digits, fraction digits, and exponent, forms the significand, strips leading and trailing zeros (tracking the decimal point), and re-emits a plain decimal. Thus 4.0, 4.50e1, and 4 all canonicalize to 4; 0.10 to 0.1; 1e3 to 1000; and a thirty-digit integer survives exactly. Two consequences matter. First, the result is lossless for big integers and high-precision decimals. Second, and central to this paper, the result is *independent of any language’s float printer*: because no float is involved, the Python and Nim implementations agree by construction rather than by luck.

Object member order. Members are sorted ascending by Unicode code point—the sequence of scalar values—which is deterministic and identical across implementations. (UTF-8 byte order coincides with code-point order, so a byte-wise sort suffices.) This differs from JCS, which sorts by UTF-16 code unit; the two agree except for astral-plane keys, and the JCS ordering is available in JCS mode (Section 7).

Strings. Every character is emitted as raw UTF-8 except those that must be escaped: ", \, the five short escapes, and other C0 controls as lowercase \u00xx. Non-ASCII characters and / are never escaped, which erases all `ensure_ascii` differences between producers.

The input grammar is a deliberate superset of RFC 8259: BOM autodetection; JSON5 features (unquoted Unicode/escaped identifier keys, single quotes, comments, trailing commas, hex and leading-plus numbers, extra string escapes, line continuations); Python literals (`True/False/None`); and non-finite tokens. Every non-standard feature is normalized away on output and, in lint mode, reported.

5 Architecture and the Byte-Parity Contract

Both implementations share one internal shape: a lenient parser produces a value tree (null, bool, number-as-raw-token, string, array, object, non-finite), and a serializer walks the tree to produce canonical bytes. Numbers are carried as their raw token text and normalized only at serialization time, which is what makes the decimal-string scheme above possible.

The contract is: *for the same input and options, the Python and Nim CLIs must emit identical bytes*. `SPEC.md` is the arbiter; a disagreement is by definition a bug in at least one implementation, never a matter of taste. The contract is enforced by `tools/parity.sh`, which runs both implementations over a fixture corpus and an option matrix (number formats, output encodings with and without BOM, lint, JCS, the binary formats, the validators, the transforms) and asserts byte-identical results against each other and against committed golden files. A second script, `tools/check.sh`, wraps the parity gate together with static analysis (`mypy --strict`, `pyright`, `nim check`) and both unit-test suites. Continuous integration runs the whole gate, plus the fuzzers of Section 8, on every change.

This architecture turns specification ambiguity into a mechanical signal. When the specification is silent or unclear, the two implementations tend to diverge, the gate fails, and the ambiguity must be resolved in `SPEC.md` before either implementation is “right.”

6 Determinism by Construction

Byte parity across two languages is not free; several techniques are required to prevent each language’s own libraries from introducing drift.

Numbers without floats. As above, decimal-string normalization removes the single largest source of cross-language disagreement—each language’s float-to-string routine—from the default path entirely.

Generated lookup tables. Where data is unavoidable, it is generated once and embedded *identically* in both languages rather than computed from each language’s standard library. Two cases: (i) the Ryu shortest-float core needs two 128-bit power-of-five tables (326 and 342 entries); these are produced by a generator script (`tools/gen_ryu_tables.py`) from exact big-integer arithmetic and emitted as a Nim source file, with the generation formulas anchored against published constants. (ii) JSON5 unquoted-key classification follows the ECMAScript `IdentifierName` grammar, which depends on Unicode general categories; because Python’s `unicodedata` and Nim’s `std/unicode` can differ by Unicode version, we generate `ID_Start`/`ID_Continue` code-point ranges once (pinned to one Unicode version) and embed the same table in both, so classification cannot drift.

A shared regular-expression subset. JSON Schema’s `pattern` keyword requires regular-expression matching. Delegating to each language’s regex engine would break parity on edge cases, so we implement one small backtracking regex engine—literals, `.`, character classes, anchors, groups, alternation, the standard quantifiers, and the common escapes—with identical semantics in both languages, and validate it externally (Section 8).

Per-code-point processing. Operations that index “characters”—UTF-16 key ordering for JCS, string length bounds for schema validation, identifier scanning—are defined on Unicode code points, matching Python’s native string indexing, and the Nim port decodes to code points rather than operating on UTF-8 bytes where it would differ.

Pure SHA-256. Because Nim’s standard library ships no SHA-256, the `--sha256` content hash uses a dependency-free in-tree implementation, validated against Python’s `hashlib`; this keeps the digest a function of the canonical bytes alone, with no third-party variation.

7 The Lossless/JCS Tension

The lossless decimal default and RFC 8785 are in direct conflict on numbers. JCS mandates that each number be serialized as ECMAScript’s `Number.prototype.toString`: parse to IEEE-754 binary64 and print the shortest round-tripping decimal. This necessarily rounds: the integer 9007199254740993 becomes 9007199254740992, and values beyond the binary64 range are not representable at all. The lossless engine, by contrast, would preserve 9007199254740993 exactly.

JSONCANON treats this as a first-class design decision rather than a silent behavior. The lossless engine is the default. JCS is an opt-in mode (`--jcs`) that switches both the number serializer (to a from-scratch port of Ryu’s double-to-string, followed by the ECMAScript formatting thresholds) and the key ordering (to UTF-16 code units). When JCS rounds or overflows a value, the tool emits a warning naming the JSON path and the exact before/after, e.g. 9007199254740993 -> 9007199254740992 (IEEE-754 rounding; not reversible). The warning cannot be silently

discarded: the quiet flag that suppresses it on standard error *requires* a log destination, so a meaning-changing conversion always leaves a trace. This “warn, never silently lose” discipline is, we argue, the right default for a tool whose entire purpose is that the output faithfully represents the input.

8 Validation Methodology

Byte parity establishes that the two implementations *agree*. It does not establish that they are *correct*: a shared misreading of the specification could make both wrong in the same way. We address this by validating the genuinely hard components against external reference implementations, and by fuzzing at scale. All harnesses are in-tree and reproducible.

Shortest-float vs. a JavaScript engine. The Ryu port is the riskiest numeric component. We feed the bit patterns of several million random doubles (plus hard edge cases: subnormals, the maximum double, every exponent-threshold boundary) to the Nim implementation and compare each output, byte for byte, to Node.js’s `String(x)`—the very function RFC 8785 references. We have run this to ten million doubles with zero mismatches. Separately, a cross-language harness feeds the same number tokens to both CLIs in JCS mode and confirms identical output, exercising the decimal-to-double parse on both sides.

Regex subset vs. CPython. The shared regex engine is validated two ways simultaneously over tens of thousands of randomly generated pattern/text pairs: each result is compared against CPython’s `re.search` (correctness for the supported subset) and against the Nim port (cross-language parity). Both held at forty thousand pairs.

Half-precision floats, exhaustively. The CBOR encoder selects the shortest of float16/32/64. The float16 codec is verified *exhaustively* against Python’s `struct` over all 2^{16} bit patterns.

Salvage mode, by mutation. The best-effort recovery mode (`--force`) is fuzzed over thousands of randomly corrupted documents, asserting that both implementations produce identical recovered output *and* identical recovery reports. This fuzzer surfaced a latent parity bug—malformed number tokens that the lenient parser accepted but the serializer later rejected with differing error messages—which we fixed by validating number tokens at parse time in both languages.

The committed gate. Beyond fuzzing, the parity gate asserts fifty-eight byte-identical outcomes across fixtures and the option/format/validator/transform matrix, and both unit-test suites pass under strict static analysis. Table 2 summarizes.

9 Feature Surface

On top of canonicalization, the same value-tree substrate supports a range of JSON-adjacent operations, each implemented twice and held to the same parity contract. Table 3 lists them. Three categories are worth noting. *Validators* report conformance issues at a JSON path: I-JSON (RFC 7493), GeoJSON (RFC 7946), JTD (RFC 8927), CDDL (RFC 8610, a practical subset), and JSON Schema (Draft 2020-12 and Draft-07, including `$ref/$anchor`, the applicators, `unevaluated*`, and opt-in `format` assertion). *Cross-format* conversion uses deterministic

Table 2: Validation harnesses and their oracles.

Component	Oracle	Scale
Ryu shortest float	Node.js <code>String(x)</code>	10^7 random doubles
JCS number tokens	cross-impl (Nim==Python)	5×10^5 tokens
Regex subset	CPython <code>re</code> and cross-impl	4×10^4 pairs
CBOR float16 codec	Python <code>struct</code>	all 2^{16} patterns
<code>--force</code> salvage	cross-impl (output + report)	4×10^3 mutations
Full canonical form	cross-impl + golden files	58 parity assertions

Table 3: Capabilities, all dual-implemented and parity-checked.

Area	Capability	Reference
Canonical form	lossless decimal numbers, code-point key sort	this work
	JCS mode (Ryu numbers, UTF-16 sort)	RFC 8785
Input dialects	JSON5 keys/escapes/numbers	JSON5
	NDJSON; JSON text sequences; concatenated	RFC 7464
	<code>--force</code> salvage of malformed input	this work
Cross-format	CBOR <code>--from/--to</code>	RFC 8949
	MessagePack <code>--from/--to</code>	MessagePack
	JData N-D arrays <code>--from jdata</code>	NeuroJSON
Validation	I-JSON; GeoJSON	RFC 7493/7946
	JTD; CDDL (subset)	RFC 8927/8610
	JSON Schema 2020-12/draft-07 (<code>+--format</code>)	JSON Schema
Transforms	JSON Pointer; Patch; Merge Patch	RFC 6901/6902/7386
Utility	lint; <code>--check</code> ; <code>--sha256</code> ; <code>--diff</code>	this work
	output re-encoding (UTF-8/16/32, latin-1, BOM)	this work

encodings: CBOR core deterministic encoding (RFC 8949 §4.2) and an analogous deterministic MessagePack, both round-tripped and byte-checked across implementations. *Transforms* run before canonicalization: JSON Pointer extraction, JSON Patch, and JSON Merge Patch.

10 Limitations

We state the boundaries of the method plainly.

- **Parity is agreement, not correctness.** Two implementations can share a misreading of the specification and agree on the wrong output. This is precisely why the hard components are validated against *external* oracles (Node.js, CPython’s `re`, Python `struct`); for components without an external oracle, confidence rests on the specification and the test suite, as in any project.
- **No machine-checked proofs.** Correctness is established empirically (differential fuzzing) and by review, not by formal verification.
- **Deliberate subsets.** The CDDL validator covers common constructs, not the full grammar (no generics, sockets, or control operators); JSON Schema omits remote `$ref` and `$dynamicRef` (which require network resolution / dynamic scope) and the `content*` keywords, and its `unevaluated*` support covers the common applicators rather than the full recursive annotation algorithm. The bundled regex engine is a subset of ECMA-262.

- **In-memory model.** Documents are parsed into a value tree; multi-gigabyte single documents are out of scope (record-oriented NDJSON and JSON text sequences are already processed incrementally).
- **Lossy by request.** JCS, CBOR, and MessagePack route numbers through binary64 (and integers beyond 64 bits, for MessagePack, through a float); this is inherent to those formats and is reported rather than hidden.

11 Conclusion

JSONCANON is, to our knowledge, the first self-contained, dependency-free toolkit to span the full JSON-integrity lifecycle in a single artifact, and it demonstrates that a canonicalization tool can be built around a correctness discipline stronger than single-implementation testing: two independent implementations forced to agree byte-for-byte through a written specification, with the genuinely hard components cross-checked against external reference implementations. The engineering required to make two languages agree exactly—decimal-only number normalization, generated lookup tables shared across languages, a shared regex subset, strict per-code-point processing—is reusable beyond this project. The same substrate cleanly supports a broad surface of JSON-adjacent functionality (dialect ingestion, validation against four schema languages, deterministic binary formats, and structural transforms) without compromising the parity contract. We believe the dual-implementation, externally-oracled approach is a good fit for any primitive whose value depends on independent parties computing the same bytes.

Availability and reproduction. The two implementations, the specification, the golden fixtures, the validation harnesses, and the continuous-integration configuration are released together at <https://github.com/lucianofedericopereira/json-canon>. A single script, `tools/reproduce.sh`, rebuilds both implementations and runs the full gate followed by every differential fuzzer (each with a fixed seed) against its external oracle, reproducing Table 2 end to end; `tools/reproduce.sh quick` runs the same pipeline at reduced scale for a fast check. Every quantitative claim in this paper is reproducible by that one command.

References

- U. Adams. Ryū: Fast float-to-string conversion. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 270–282. ACM, 2018. doi: 10.1145/3192366.3192369.
- H. Birkholz, C. Vigano, and C. Bormann. Concise data definition language (CDDL). Technical Report RFC 8610, IETF, 2019. URL <https://www.rfc-editor.org/rfc/rfc8610>.
- C. Bormann and P. E. Hoffman. Concise binary object representation (CBOR). Internet Standard (STD 94) RFC 8949, IETF, 2020. URL <https://www.rfc-editor.org/rfc/rfc8949>.
- T. Bray. The I-JSON message format. Technical Report RFC 7493, IETF, 2015. URL <https://www.rfc-editor.org/rfc/rfc7493>.
- T. Bray. The JavaScript Object Notation (JSON) data interchange format. Internet Standard (STD 90) RFC 8259, IETF, 2017. URL <https://www.rfc-editor.org/rfc/rfc8259>.

- P. C. Bryan and M. Nottingham. JavaScript Object Notation (JSON) patch. Technical Report RFC 6902, IETF, 2013. URL <https://www.rfc-editor.org/rfc/rfc6902>.
- P. C. Bryan, K. Zyp, and M. Nottingham. JavaScript Object Notation (JSON) pointer. Technical Report RFC 6901, IETF, 2013. URL <https://www.rfc-editor.org/rfc/rfc6901>.
- H. Butler, M. Daly, A. Doyle, S. Gillies, S. Hagen, and T. Schaub. The GeoJSON format. Technical Report RFC 7946, IETF, 2016. URL <https://www.rfc-editor.org/rfc/rfc7946>.
- U. Carion. JSON type definition. Technical Report RFC 8927, IETF, 2020. URL <https://www.rfc-editor.org/rfc/rfc8927>.
- Q. Fang. JData: A general-purpose data annotation and interchange format. NeuroJSON, 2015. URL <https://neurojson.org/>.
- S. Furuhashi. MessagePack specification, 2008. URL <https://github.com/msgpack/msgpack/blob/master/spec.md>.
- P. E. Hoffman and J. M. Snell. JSON merge patch. Technical Report RFC 7386, IETF, 2014. URL <https://www.rfc-editor.org/rfc/rfc7386>.
- M. B. Jones, J. Bradley, and N. Sakimura. JSON web signature (JWS). Technical Report RFC 7515, IETF, 2015. URL <https://www.rfc-editor.org/rfc/rfc7515>.
- A. Kishore, M. Nanasy, A. Eisenberg, et al. JSON5 data interchange format, 2023. URL <https://json5.org/>.
- W. M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.
- W. McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference (SciPy)*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.
- A. Rundgren, B. Jordan, and S. Erdtman. JSON canonicalization scheme (JCS). Technical Report RFC 8785, IETF, 2020. URL <https://www.rfc-editor.org/rfc/rfc8785>.
- N. Williams. JavaScript Object Notation (JSON) text sequences. Technical Report RFC 7464, IETF, 2015. URL <https://www.rfc-editor.org/rfc/rfc7464>.
- A. Wright, H. Andrews, and B. Hutton. JSON schema: A media type for describing JSON documents, draft 2020-12, 2022. URL <https://json-schema.org/specification>.
- X. Yang, Y. Chen, E. Eide, and J. Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 283–294. ACM, 2011. doi: 10.1145/1993498.1993532.